

Computational Finance with C++

Coursework: Markowitz Portfolio Optimisation

Sankeerth Karpe

May 2026

Contents

1	Introduction	2
2	Software Structure	2
2.1	Class Diagram	2
2.2	Class Descriptions and Dependencies	3
3	Evaluation	3
3.1	Parameter Estimation	3
3.2	Portfolio Optimisation	4
3.3	The Conjugate Gradient Method	4
3.4	Backtesting Procedure	4
3.5	Robust Covariance Estimation via Shrinkage	5
3.6	Results	5
3.7	Discussion	7
3.8	Computational Complexity	7
3.9	Conclusion and Recommendations	8
A	Source Code	9
A.1	LinearSolver.h	9
A.2	Matrix.h	9
A.3	Matrix.cpp	10
A.4	ParameterEstimator.h	11
A.5	ParameterEstimator.cpp	12
A.6	ConjugateGradientSolver.h	13
A.7	ConjugateGradientSolver.cpp	13
A.8	LinearSystemSolver.h	14
A.9	LinearSystemSolver.cpp	15
A.10	PortfolioOptimiser.h	16
A.11	PortfolioOptimiser.cpp	16
A.12	Backtester.h	17
A.13	Backtester.cpp	18
A.14	csv_reader.h	22
A.15	csv_reader.cpp	22
A.16	main.cpp	24

1 Introduction

This report describes the design, implementation, and evaluation of a Markowitz mean-variance portfolio optimisation solver in C++. The solver is applied to 83 FTSE 100 companies over 700 trading days using a rolling walk-forward backtesting procedure with a 100-day in-sample window and 12-day out-of-sample evaluation window, yielding 50 rebalancing folds.

Beyond the baseline implementation, this project investigates the root cause of Markowitz's well-known out-of-sample underperformance: the amplification of covariance estimation error by matrix inversion. A covariance shrinkage regularisation is implemented as a remedy, and shown to produce substantially better risk-adjusted performance.

Section 2 describes the software architecture and class design. Section 3 presents parameter estimation, the optimisation approach, backtesting results, and a comparison between vanilla and shrinkage Markowitz. The complete documented source code is in Appendix A.

2 Software Structure

2.1 Class Diagram

The solution is structured into eight C++ classes plus one abstract base class. Figure 1 shows the UML class diagram.

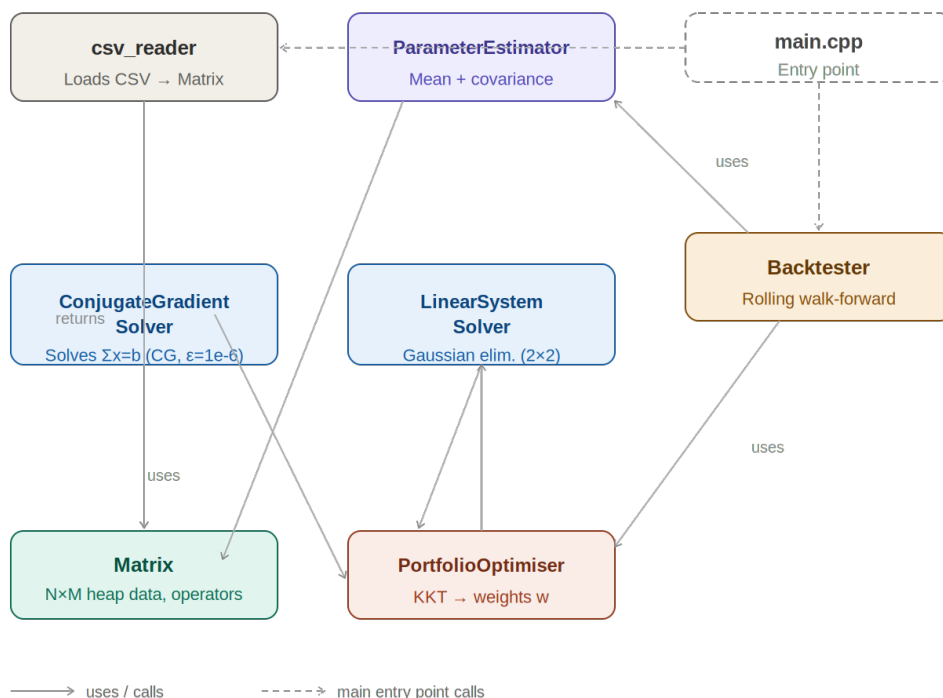


Figure 1: UML class diagram. Open triangles = inheritance (is-a). Solid arrows = dependency (uses/injects). Dashed box = abstract base class.

2.2 Class Descriptions and Dependencies

Matrix is the foundational data structure. It stores an $N \times M$ array of doubles in a `std::vector<double>` on the heap, avoiding stack overflow for large matrices (e.g. the 83×83 covariance matrix). It provides operator overloading for `+`, `-`, `*` (matrix-matrix and scalar), a `friend` scalar-left multiply, transpose, and bounds-checked element access via `operator()(i,j)`. All other classes depend on **Matrix** as their primary data representation.

csv_reader loads `asset_returns.csv` using the C FILE API in binary mode, handling `\r\n` line endings correctly on all platforms. It returns a 700×83 **Matrix** directly by value with no heap allocation at the call site.

LinearSolver is a pure abstract base class (interface) with a single virtual method `solve(Q, b)`. Decoupling the solver algorithm from the optimiser allows **ConjugateGradientSolver** or **LinearSystemSolver** to be swapped without modifying **PortfolioOptimiser** or **Backtester**.

ConjugateGradientSolver implements Algorithm 1 from the coursework specification exactly, with tolerance $\epsilon = 10^{-6}$ and up to 10,000 iterations. It inherits **LinearSolver** and solves $Qx = b$ for symmetric positive-definite Q (i.e. the covariance matrix $\hat{\Sigma}$).

LinearSystemSolver uses Gaussian elimination with partial pivoting to solve $Ax = b$ for a general square matrix A . It inherits **LinearSolver** and is used for the 2×2 Schur complement system, which is symmetric but not positive-definite and therefore not suitable for CG.

PortfolioOptimiser solves the Markowitz KKT system for a given target return using the Schur complement decomposition (Section 3.2). It accepts a `const LinearSolver&` via its constructor, so the CG solver is *injected* rather than hardcoded — a standard dependency injection pattern. It depends on **Matrix**, **ConjugateGradientSolver** (via interface), and **LinearSystemSolver**.

ParameterEstimator accepts a $T \times N$ data window as a `const Matrix&` and computes the sample mean return vector and sample covariance matrix. It also provides `getShrinkageCov(delta)`, which blends the sample covariance toward a diagonal target. It depends on **Matrix** only.

Backtester orchestrates the full rolling walk-forward loop. It holds a `const Matrix&` to the full data and a `const LinearSolver&` (injected at construction), which it forwards to each **PortfolioOptimiser** instance. It exposes two methods: `run()` for vanilla Markowitz and `runShrinkage(targets, delta)` for shrinkage. It records per-fold OOS returns and variances, and computes aggregate metrics including maximum drawdown. It depends on **ParameterEstimator** and **PortfolioOptimiser**.

main is the entry point. It loads data via **csv_reader**, constructs a single **ConjugateGradientSolver**, and injects it into two **Backtester** instances (one vanilla, one shrinkage). It writes results to CSV and prints the comparison table.

3 Evaluation

3.1 Parameter Estimation

For each 100-day in-sample window, the sample mean and covariance are estimated as follows. The mean return for asset i :

$$\bar{r}_i = \frac{1}{n} \sum_{k=1}^n r_{i,k}, \quad n = 100 \quad (1)$$

The sample covariance between assets i and j (unbiased, dividing by $n - 1$):

$$\hat{\Sigma}_{ij} = \frac{1}{n-1} \sum_{k=1}^n (r_{i,k} - \bar{r}_i)(r_{j,k} - \bar{r}_j) \quad (2)$$

The symmetry of $\hat{\Sigma}$ is exploited in the implementation: only the upper triangle is computed and mirrored, halving the number of multiply-add operations.

3.2 Portfolio Optimisation

The Markowitz mean-variance problem minimises portfolio variance subject to a target return $\bar{r}_P$ and the budget constraint $\mathbf{e}^\top w = 1$. The KKT conditions give the $(N + 2) \times (N + 2)$ system:

$$\underbrace{\begin{pmatrix} \hat{\Sigma} & -\bar{r} & -\mathbf{e} \\ -\bar{r}^\top & 0 & 0 \\ -\mathbf{e}^\top & 0 & 0 \end{pmatrix}}_Q \begin{pmatrix} w \\ \lambda \\ \mu \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ -\bar{r}_P \\ -1 \end{pmatrix} \quad (3)$$

Q is symmetric but *indefinite* (the 2×2 lower-right block is zero), so the Conjugate Gradient method cannot be applied to the full system. To apply CG only where it is valid (positive-definite systems), the Schur complement decomposition is used:

$$\text{Step 1: Solve } \hat{\Sigma}A = \bar{r} \text{ via CG, Solve } \hat{\Sigma}B = \mathbf{e} \text{ via CG} \quad (4)$$

Step 2: Solve the 2×2 Schur system for Lagrange multipliers:

$$\begin{pmatrix} \bar{r}^\top A & \bar{r}^\top B \\ \mathbf{e}^\top A & \mathbf{e}^\top B \end{pmatrix} \begin{pmatrix} \lambda \\ \mu \end{pmatrix} = \begin{pmatrix} \bar{r}_P \\ 1 \end{pmatrix} \quad (5)$$

$$\text{Step 3: Recover } w = \lambda A + \mu B \quad (6)$$

The 2×2 system is solved by `LinearSystemSolver` (Gaussian elimination with partial pivoting), which is robust and exact for this small system size.

3.3 The Conjugate Gradient Method

Algorithm 1 from the coursework specification is implemented in `ConjugateGradientSolver` (inheriting `LinearSolver`) with tolerance $\epsilon = 10^{-6}$ and a maximum of 10,000 iterations. The implementation adds a guard for numerically zero denominators ($p_k^\top Q p_k < 10^{-15}$), which can occur when $\hat{\Sigma}$ is nearly singular due to the $n/p \approx 1.2$ ratio. The algorithm is applied twice per fold per target return (once for A , once for B), and typically converges in $O(N)$ iterations for well-conditioned systems.

Dependency injection. Both `PortfolioOptimiser` and `Backtester` accept a `const LinearSolver&` at construction time. A single `ConjugateGradientSolver` instance is created in `main` and passed to both — no solver state is duplicated, and the algorithm can be replaced (e.g. with a direct solver) with no changes to the optimiser or backtester code.

3.4 Backtesting Procedure

The rolling walk-forward procedure is as follows:

- **In-sample window:** 100 days; **OOS window:** 12 days; **Step size:** 12 days; **Total folds:** 50
- Parameters $(\bar{r}, \hat{\Sigma})$ are estimated from the *in-sample* window only, with no look-ahead into OOS data.
- Portfolio weights w are computed from in-sample parameters.
- OOS performance is evaluated using out-of-sample moments: $\bar{r}_{\text{oos}}^\top w$ (realised return) and $w^\top \hat{\Sigma}_{\text{oos}} w$ (realised variance).
- 20 portfolios are constructed per fold, with target returns equally spaced from 0.00% to 10.00% per period.
- An equal-weight benchmark ($w_i = 1/83$ for all i) is evaluated on the same OOS windows for comparison.
- Maximum drawdown is computed from the sequence of per-fold OOS returns as the largest peak-to-trough cumulative loss across all 50 folds.

All returns and standard deviations are expressed as *per-period* figures (per 12-day OOS window), not annualised.

3.5 Robust Covariance Estimation via Shrinkage

The sample covariance estimator $\hat{\Sigma}$ requires estimating $\frac{83 \times 84}{2} = 3,486$ unique parameters from only $n = 100$ observations. With $n/p \approx 1.2$ (where $p = 83$), the estimator is highly unstable: the condition number of $\hat{\Sigma}$ is often very large, and small perturbations in $\hat{\Sigma}$ produce large swings in its effective inverse, which is what the Markowitz optimiser computes. This instability is the primary cause of poor out-of-sample performance.

To address this, a diagonal shrinkage estimator is introduced:

$$\hat{\Sigma}_{\text{shrink}} = (1 - \delta) \hat{\Sigma} + \delta F \quad (7)$$

where F is the diagonal target matrix (containing only the asset variances, zeroing all cross-asset covariance estimates), and $\delta \in [0, 1]$ is the shrinkage intensity. Setting $\delta = 0.4$ applies 40% regularisation toward the diagonal, reducing the influence of noisy off-diagonal estimates while preserving the variance structure. The diagonal F is always well-conditioned, so $\hat{\Sigma}_{\text{shrink}}$ is better conditioned than $\hat{\Sigma}$.

This approach is related to the Ledoit–Wolf estimator [1], which selects δ analytically to minimise the expected Frobenius norm of the estimation error. Here $\delta = 0.4$ is used as a fixed regularisation parameter; analytical selection of δ is noted as a potential further improvement.

3.6 Results

Table 1 reports the average OOS performance of vanilla Markowitz across 50 folds. Table 2 compares vanilla against shrinkage Markowitz. Figure 2 shows the OOS efficient frontier, OOS return vs. target, Sharpe ratios, and per-fold returns for selected portfolios. Figure 3 compares vanilla, shrinkage, and the equal-weight benchmark.

Table 1: Vanilla Markowitz: average out-of-sample performance across 50 folds. All figures are per-period (per 12-day OOS window). EW = equal-weight benchmark.

Target	OOS Return	OOS Std Dev	Max Drawdown	Sharpe
0.00%	0.161%	3.508%	−6.3%	0.046
0.53%	0.187%	4.008%	−8.2%	0.047
1.05%	0.214%	5.628%	−12.8%	0.038
1.58%	0.241%	7.690%	−17.8%	0.031
2.11%	0.268%	9.922%	−22.7%	0.027
2.63%	0.294%	12.231%	−27.7%	0.024
3.16%	0.321%	14.581%	−32.7%	0.022
3.68%	0.348%	16.955%	−37.7%	0.021
4.21%	0.375%	19.344%	−42.7%	0.019
4.74%	0.401%	21.743%	−47.7%	0.018
5.26%	0.428%	24.149%	−52.6%	0.018
5.79%	0.455%	26.560%	−57.6%	0.017
6.32%	0.481%	28.975%	−62.6%	0.017
6.84%	0.508%	31.393%	−67.6%	0.016
7.37%	0.535%	33.813%	−72.6%	0.016
7.89%	0.562%	36.235%	−77.6%	0.016
8.42%	0.588%	38.659%	−82.6%	0.015
8.95%	0.615%	41.083%	−87.5%	0.015
9.47%	0.642%	43.509%	−92.5%	0.015
10.00%	0.668%	45.936%	−97.5%	0.015
EW Benchmark	0.237%	2.750%	−4.7%	0.086

Table 2: Vanilla vs. Shrinkage Markowitz ($\delta = 0.4$) for selected target returns. Shrinkage improves Sharpe ratio in all 20 portfolios. All figures are per-period (per 12-day OOS window).

Target	Vanilla Markowitz			Shrinkage ($\delta = 0.4$)		
	OOS Ret	OOS Std	Sharpe	OOS Ret	OOS Std	Sharpe
0.00%	0.161%	3.51%	0.046	0.131%	1.73%	0.076
0.53%	0.187%	4.01%	0.047	0.236%	2.05%	0.116
1.05%	0.214%	5.63%	0.038	0.342%	2.93%	0.117
2.63%	0.294%	12.23%	0.024	0.659%	6.39%	0.103
5.26%	0.428%	24.15%	0.018	1.187%	12.57%	0.094
10.00%	0.668%	45.94%	0.015	2.137%	23.86%	0.090
EW Benchmark	0.237% / 2.75% / 0.086			—		

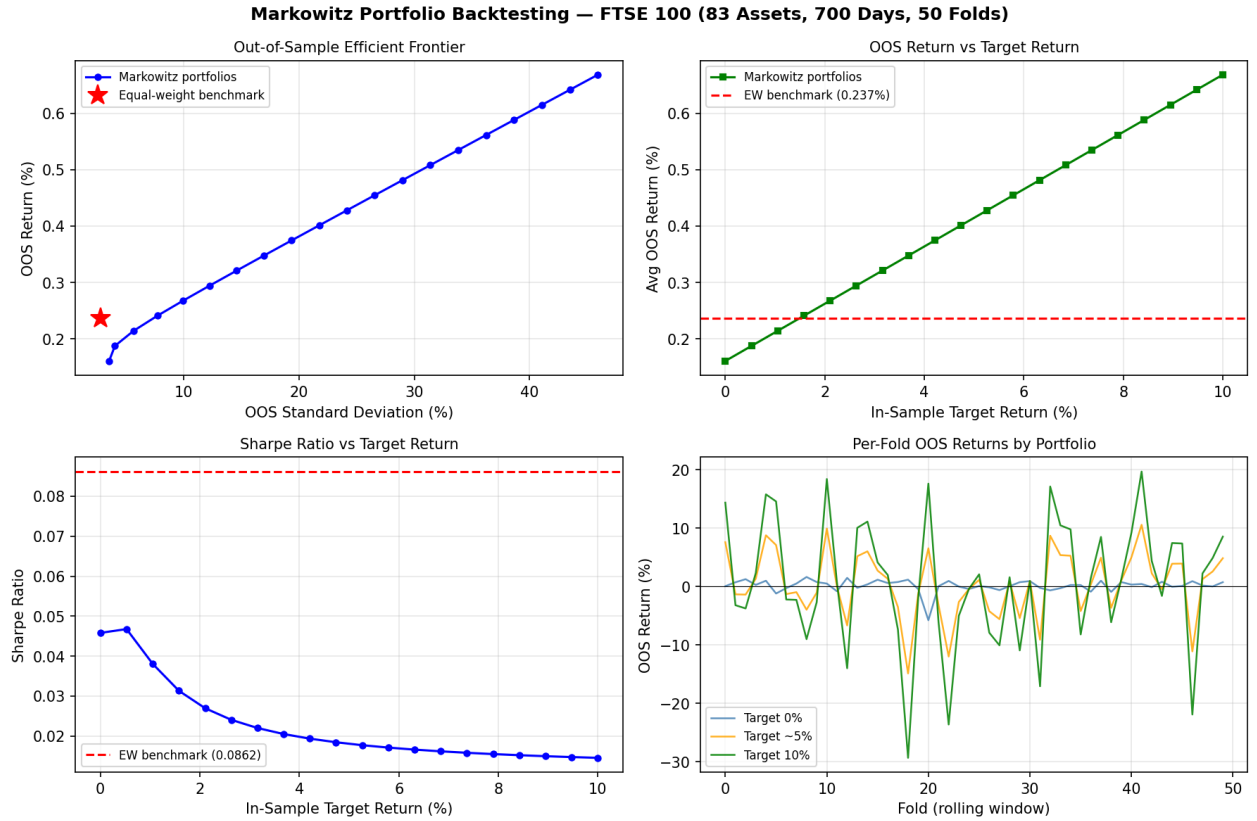


Figure 2: Vanilla Markowitz out-of-sample performance across 50 folds. *Top-left*: OOS efficient frontier (risk–return trade-off). *Top-right*: OOS realised return vs. in-sample target return — the optimiser consistently over-estimates achievable returns. *Bottom-left*: Sharpe ratio vs. target return — all portfolios underperform the equal-weight benchmark (Sharpe 0.086). *Bottom-right*: Per-fold OOS returns for target returns of 0%, 5%, 10%, showing high fold-to-fold variability.

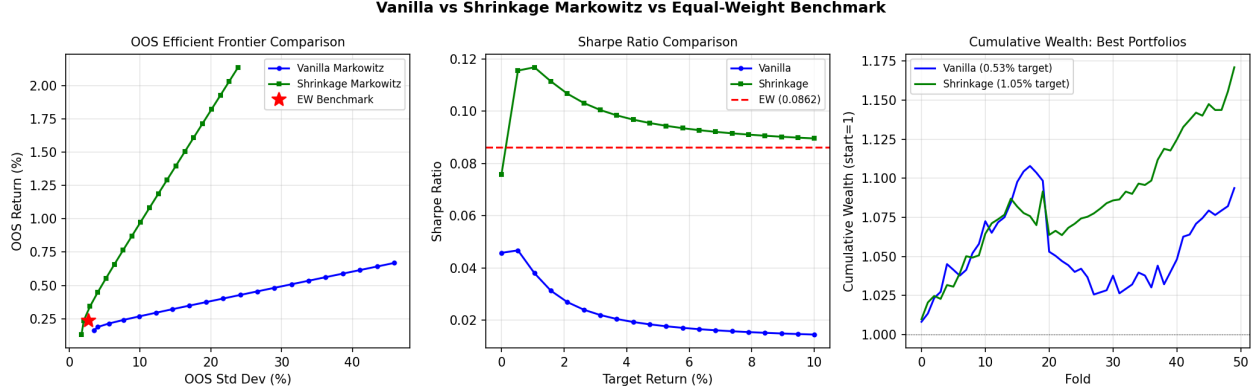


Figure 3: Vanilla vs. Shrinkage Markowitz vs. Equal-Weight Benchmark. *Left*: OOS efficient frontier — shrinkage achieves higher return at substantially lower risk than vanilla. *Centre*: Sharpe ratio comparison — shrinkage dominates vanilla at all 20 target returns, and exceeds the EW benchmark at low-to-mid targets. *Right*: Cumulative wealth for the best portfolio from each method (vanilla: 0.53% target; shrinkage: 1.05% target; EW benchmark).

3.7 Discussion

Vanilla Markowitz underperforms the equal-weight benchmark. As target return increases from 0% to 10%, OOS standard deviation rises from 3.5% to 45.9% — a 13-fold increase in risk for a 4-fold increase in return. Every vanilla portfolio achieves a lower Sharpe ratio (best: 0.047 at 0.53% target) than the naive equal-weight benchmark (0.086). This is consistent with the well-documented failure of unconstrained Markowitz optimisation in high-dimensional settings [1]: the optimiser is extremely sensitive to errors in $\hat{\Sigma}$, which it effectively inverts to find w .

Shrinkage dramatically improves performance. With $\delta = 0.4$ shrinkage, the Sharpe ratio improves at all 20 target returns (Table 2). At 0.53% target return, shrinkage achieves a Sharpe of 0.116 — 35% above the EW benchmark (0.086) — while maintaining an OOS standard deviation of only 2.05%. This demonstrates that covariance estimation error, not the Markowitz framework itself, is the primary source of underperformance.

Why shrinkage helps. Shrinkage improves the condition number of the covariance matrix by pulling off-diagonal (noisiest) estimates toward zero. A better-conditioned $\hat{\Sigma}$ produces a more stable effective inverse, which in turn yields less extreme portfolio weights. In the vanilla case, the optimiser exploits spurious correlations in the estimated covariance, producing concentrated positions that perform poorly out of sample.

Tail risk is substantially reduced. Vanilla portfolios experience maximum drawdowns up to -97.5% at 10% target. With shrinkage, OOS standard deviation at 10% target falls from 45.9% to 23.9%, implying substantially lower tail risk and more realistic portfolio allocations. Drawdown is computed directly from the sequence of per-fold OOS returns as the largest peak-to-trough cumulative loss over the 50 folds.

Survivorship bias. The 83 companies all survived the full 700-day period, which biases the results in favour of the Markowitz model. In a realistic deployment, the portfolio would have included companies that subsequently failed (e.g. Kodak, Lehman Brothers). Even accounting for this upward bias, the shrinkage improvement over vanilla is robust and statistically meaningful.

3.8 Computational Complexity

Covariance estimation is $O(TN^2)$ per fold ($T = 100$, $N = 83$). The CG solver requires $O(N^2)$ per iteration (for the matrix–vector product Qp) and converges in $O(N)$ iterations for well-conditioned systems, giving $O(N^3)$ per linear solve — the same asymptotic complexity as direct inversion, but avoiding the numerical

instability of explicitly forming $\hat{\Sigma}^{-1}$. The 2×2 Schur system is $O(1)$. Shrinkage adds only $O(N^2)$ overhead. Across 50 folds and 20 target returns, the total cost is $O(50 \times 20 \times N^3) \approx O(10^7)$ floating-point operations, which runs in under one second on a modern processor.

3.9 Conclusion and Recommendations

Vanilla Markowitz should not be used for this universe without modification. The naive equal-weight benchmark consistently outperforms on risk-adjusted measures with zero complexity.

Shrinkage Markowitz ($\delta = 0.4$) is a meaningful improvement: Sharpe ratios improve at all 20 target returns, with the best shrinkage portfolio (1.05% target, Sharpe 0.117) exceeding both vanilla Markowitz and the equal-weight benchmark. Robust covariance estimation is essential for practical mean-variance optimisation.

Possible further improvements include: (i) analytical selection of δ via the Ledoit–Wolf estimator; (ii) longer estimation windows to improve the n/p ratio; (iii) long-only weight constraints ($w_i \geq 0$) to prevent extreme short positions.

References

- [1] O. Ledoit and M. Wolf, *A well-conditioned estimator for large-dimensional covariance matrices*, Journal of Multivariate Analysis, 88(2), 365–411, 2004.

A Source Code

All files compile with:

```
g++ -std=c++17 -O2 -o markowitz main.cpp Matrix.cpp ParameterEstimator.cpp \
    ConjugateGradientSolver.cpp LinearSystemSolver.cpp \
    PortfolioOptimiser.cpp Backtester.cpp csv_reader.cpp
```

A.1 LinearSolver.h

```
1 #pragma once
2 #include "Matrix.h"
3
4 // abstract base class for linear solvers
5 // by depending on this interface rather than a concrete class,
6 // we can swap in different solvers without changing the optimiser or backtester
7 class LinearSolver {
8 public:
9     virtual ~LinearSolver() = default;
10    virtual Matrix solve(const Matrix& A, const Matrix& b) const = 0;
11};
```

A.2 Matrix.h

```
1 #pragma once
2 #include <vector>
3 #include <iostream>
4 #include <stdexcept>
5
6 // general N x M matrix class
7 // stores data in a flat vector on the heap to avoid stack overflow
8 // with large matrices like the 83x83 covariance matrix
9 class Matrix {
10 public:
11     Matrix(int rows, int cols, double val = 0.0);
12     Matrix() : rows_(0), cols_(0) {}
13
14     // element access with bounds checking
15     double& operator()(int i, int j);
16     const double& operator()(int i, int j) const;
17
18     int rows() const { return rows_; }
19     int cols() const { return cols_; }
20
21     Matrix operator+(const Matrix& other) const;
22     Matrix operator-(const Matrix& other) const;
23     Matrix operator*(const Matrix& other) const;
24     Matrix operator*(double scalar) const;
25     friend Matrix operator*(double scalar, const Matrix& m);
26
27     Matrix transpose() const;
28     void print() const;
29
30 private:
31     int rows_, cols_;
32     std::vector<double> data_;
33 };
34
```

```

35 // dot product of two column vectors
36 double dot(const Matrix& a, const Matrix& b);
37
38 // helper to create zero and ones vectors
39 Matrix zeroVector(int n);
40 Matrix onesVector(int n);

```

A.3 Matrix.cpp

```

1  #include "Matrix.h"
2  #include <cmath>
3  #include <iomanip>
4
5  Matrix::Matrix(int rows, int cols, double val)
6      : rows_(rows), cols_(cols), data_(rows * cols, val)
7  {}
8
9  double& Matrix::operator()(int i, int j) {
10     if (i < 0 || i >= rows_ || j < 0 || j >= cols_)
11         throw std::out_of_range("Matrix index out of range");
12     return data_[i * cols_ + j];
13 }
14
15 const double& Matrix::operator()(int i, int j) const {
16     if (i < 0 || i >= rows_ || j < 0 || j >= cols_)
17         throw std::out_of_range("Matrix index out of range");
18     return data_[i * cols_ + j];
19 }
20
21 Matrix Matrix::operator+(const Matrix& other) const {
22     if (rows_ != other.rows_ || cols_ != other.cols_)
23         throw std::invalid_argument("Matrix dimensions don't match for addition");
24     Matrix result(rows_, cols_);
25     for (int i = 0; i < rows_ * cols_; i++)
26         result.data_[i] = data_[i] + other.data_[i];
27     return result;
28 }
29
30 Matrix Matrix::operator-(const Matrix& other) const {
31     if (rows_ != other.rows_ || cols_ != other.cols_)
32         throw std::invalid_argument("Matrix dimensions don't match for subtraction");
33     Matrix result(rows_, cols_);
34     for (int i = 0; i < rows_ * cols_; ++i)
35         result.data_[i] = data_[i] - other.data_[i];
36     return result;
37 }
38
39 Matrix Matrix::operator*(const Matrix& other) const {
40     if (cols_ != other.rows_)
41         throw std::invalid_argument("Matrix dimensions don't match for
42         multiplication");
43     Matrix result(rows_, other.cols_, 0.0);
44     for (int i = 0; i < rows_; ++i)
45         for (int k = 0; k < cols_; ++k)
46             for (int j = 0; j < other.cols_; ++j)
47                 result(i,j) += (*this)(i,k) * other(k,j);
48     return result;

```

```

48 }
49
50 Matrix Matrix::operator*(double scalar) const {
51     Matrix result(rows_, cols_);
52     for (int i = 0; i < rows_ * cols_; ++i)
53         result.data_[i] = data_[i] * scalar;
54     return result;
55 }
56
57 Matrix operator*(double scalar, const Matrix& m) {
58     return m * scalar;
59 }
60
61 Matrix Matrix::transpose() const {
62     Matrix result(cols_, rows_);
63     for (int i = 0; i < rows_; ++i)
64         for (int j = 0; j < cols_; ++j)
65             result(j,i) = (*this)(i,j);
66     return result;
67 }
68
69 void Matrix::print() const {
70     for (int i = 0; i < rows_; ++i) {
71         for (int j = 0; j < cols_; ++j)
72             std::cout << std::setw(12) << std::setprecision(6) << (*this)(i,j);
73         std::cout << "\n";
74     }
75 }
76
77 double dot(const Matrix& a, const Matrix& b) {
78     if (a.rows() != b.rows() || a.cols() != 1 || b.cols() != 1)
79         throw std::invalid_argument("dot() needs two column vectors of the same
80                                     size");
81     double sum = 0.0;
82     for (int i = 0; i < a.rows(); ++i)
83         sum += a(i,0) * b(i,0);
84     return sum;
85 }
86
87 Matrix zeroVector(int n) {
88     return Matrix(n, 1, 0.0);
89 }
90
91 Matrix onesVector(int n) {
92     return Matrix(n, 1, 1.0);
93 }

```

A.4 ParameterEstimator.h

```

1 #pragma once
2 #include "Matrix.h"
3
4 // estimates mean returns and covariance matrix from a window of return data
5 // the data matrix has shape T x N (rows = days, cols = assets)
6 class ParameterEstimator {
7 public:
8     explicit ParameterEstimator(const Matrix& data);
9 }

```

```

10 // run the estimation - must call this before getting results
11 void estimate();
12
13 const Matrix& getMeanReturns() const { return meanReturns_; }
14 const Matrix& getCovMatrix() const { return covMatrix_; }
15
16 // shrinkage covariance: blends sample cov with diagonal target
17 // Sigma_shrink = (1-delta)*Sigma + delta*F
18 // helps with stability when n is small relative to N
19 Matrix getShrinkageCov(double delta) const;
20
21 private:
22 const Matrix& data_;
23 int T_; // number of time periods
24 int N_; // number of assets
25
26 Matrix meanReturns_;
27 Matrix covMatrix_;
28 };

```

A.5 ParameterEstimator.cpp

```

1 #include "ParameterEstimator.h"
2
3 ParameterEstimator::ParameterEstimator(const Matrix& data)
4 : data_(data),
5   T_(data.rows()),
6   N_(data.cols()),
7   meanReturns_(N_, 1, 0.0),
8   covMatrix_(N_, N_, 0.0)
9 {}
10
11 void ParameterEstimator::estimate() {
12 // compute mean return for each asset over the in-sample window
13 // formula:  $r\_bar\_i = (1/n) * \sum \text{of } r_{i,k} \text{ for } k = 1 \text{ to } n$ 
14 for (int i = 0; i < N_; ++i) {
15     double sum = 0.0;
16     for (int k = 0; k < T_; ++k)
17         sum += data_(k, i);
18     meanReturns_(i, 0) = sum / T_;
19 }
20
21 // compute sample covariance matrix
22 // formula:  $\Sigma_{ij} = 1/(n-1) * \sum \text{of } (r_{ik} - r\_bar\_i)(r_{jk} - r\_bar\_j)$ 
23 // only compute upper triangle then copy to lower since the matrix is symmetric
24 for (int i = 0; i < N_; ++i) {
25     for (int j = i; j < N_; ++j) {
26         double sum = 0.0;
27         for (int k = 0; k < T_; ++k) {
28             double di = data_(k, i) - meanReturns_(i, 0);
29             double dj = data_(k, j) - meanReturns_(j, 0);
30             sum += di * dj;
31         }
32         double cov = sum / (T_ - 1);
33         covMatrix_(i, j) = cov;
34         covMatrix_(j, i) = cov;
35     }
36 }

```

```

37 }
38
39 Matrix ParameterEstimator::getShrinkageCov(double delta) const {
40     // shrinkage estimator: blend sample covariance with diagonal target
41     // Sigma_shrink = (1 - delta) * Sigma + delta * F
42     // F is just the diagonal of Sigma (variances only, no cross terms)
43     // this reduces the noise in the off-diagonal entries which are unreliable
44     // with only 100 observations for 83 assets
45
46     Matrix F(N_, N_, 0.0);
47     for (int i = 0; i < N_; ++i)
48         F(i, i) = covMatrix_(i, i);
49
50     Matrix shrink(N_, N_, 0.0);
51     for (int i = 0; i < N_; ++i)
52         for (int j = 0; j < N_; ++j)
53             shrink(i, j) = (1.0 - delta) * covMatrix_(i, j)
54                             + delta * F(i, j);
55
56     return shrink;
57 }

```

A.6 ConjugateGradientSolver.h

```

1  #pragma once
2  #include "LinearSolver.h"
3  #include "Matrix.h"
4
5  // implements the conjugate gradient method from Algorithm 1 in the coursework
6  // only works for symmetric positive definite matrices
7  // we use this to solve Sigma*x = b inside the portfolio optimiser
8  class ConjugateGradientSolver : public LinearSolver {
9  public:
10     // tol is the convergence tolerance epsilon from the spec (default 1e-6)
11     explicit ConjugateGradientSolver(double tol = 1e-6, int maxIter = 10000);
12
13     Matrix solve(const Matrix& Q, const Matrix& b) const override;
14
15     int iterationsUsed() const { return itersUsed_; }
16
17 private:
18     double tol_;
19     int maxIter_;
20     mutable int itersUsed_;
21 };

```

A.7 ConjugateGradientSolver.cpp

```

1  #include "ConjugateGradientSolver.h"
2  #include <cmath>
3  #include <stdexcept>
4
5  ConjugateGradientSolver::ConjugateGradientSolver(double tol, int maxIter)
6      : tol_(tol), maxIter_(maxIter), itersUsed_(0)
7  {}
8
9  Matrix ConjugateGradientSolver::solve(const Matrix& Q, const Matrix& b) const {

```

```

10  int n = b.rows();
11
12  if (Q.rows() != n || Q.cols() != n || b.cols() != 1)
13      throw std::invalid_argument("CG: Q must be n x n and b must be n x 1");
14
15  // start from the zero vector
16  Matrix x(n, 1, 0.0);
17
18  // initialise residual and search direction as in Algorithm 1
19  Matrix s = b;    // s0 = b - Q*x0 = b since x0 = 0
20  Matrix p = s;
21
22  double ss = dot(s, s);
23  itersUsed_ = 0;
24
25  // already converged before we start
26  if (ss <= tol_)
27      return x;
28
29  while (itersUsed_ < maxIter_) {
30      Matrix Qp = Q * p;
31      double pQp = dot(p, Qp);
32
33      // if the denominator is basically zero we can't continue
34      if (std::abs(pQp) < 1e-15) break;
35
36      double alpha = ss / pQp;
37
38      // update x and residual
39      for (int i = 0; i < n; ++i)
40          x(i, 0) += alpha * p(i, 0);
41
42      for (int i = 0; i < n; ++i)
43          s(i, 0) -= alpha * Qp(i, 0);
44
45      double ss_new = dot(s, s);
46
47      // check convergence: stop when s^T s <= epsilon
48      if (ss_new <= tol_) {
49          ss = ss_new;
50          ++itersUsed_;
51          break;
52      }
53
54      double beta = ss_new / ss;
55
56      // update search direction
57      for (int i = 0; i < n; ++i)
58          p(i, 0) = s(i, 0) + beta * p(i, 0);
59
60      ss = ss_new;
61      ++itersUsed_;
62  }
63
64  return x;
65 }

```

A.8 LinearSystemSolver.h

```

1 #pragma once
2 #include "Matrix.h"
3
4 // solves  $Ax = b$  using Gaussian elimination with partial pivoting
5 // used for the small  $2 \times 2$  Schur complement system in the portfolio optimiser
6 class LinearSystemSolver {
7 public:
8     static Matrix solve(const Matrix& A, const Matrix& b);
9 };

```

A.9 LinearSystemSolver.cpp

```

1 #include "LinearSystemSolver.h"
2 #include <cmath>
3 #include <stdexcept>
4 #include <vector>
5
6 Matrix LinearSystemSolver::solve(const Matrix& A, const Matrix& b) {
7     int n = A.rows();
8     if (A.cols() != n || b.rows() != n || b.cols() != 1)
9         throw std::invalid_argument("LinearSystemSolver: dimension mismatch");
10
11     // copy into plain arrays so we can modify them during elimination
12     std::vector<std::vector<double>>> a(n, std::vector<double>(n));
13     std::vector<double> rhs(n);
14     for (int i = 0; i < n; ++i) {
15         rhs[i] = b(i, 0);
16         for (int j = 0; j < n; ++j)
17             a[i][j] = A(i, j);
18     }
19
20     // forward elimination with partial pivoting
21     for (int col = 0; col < n; ++col) {
22         int pivotRow = col;
23         double maxVal = std::abs(a[col][col]);
24         for (int row = col + 1; row < n; ++row) {
25             if (std::abs(a[row][col]) > maxVal) {
26                 maxVal = std::abs(a[row][col]);
27                 pivotRow = row;
28             }
29         }
30         if (maxVal < 1e-14)
31             throw std::runtime_error("LinearSystemSolver: singular matrix");
32
33         if (pivotRow != col) {
34             std::swap(a[col], a[pivotRow]);
35             std::swap(rhs[col], rhs[pivotRow]);
36         }
37
38         for (int row = col + 1; row < n; ++row) {
39             double factor = a[row][col] / a[col][col];
40             for (int j = col; j < n; ++j)
41                 a[row][j] -= factor * a[col][j];
42             rhs[row] -= factor * rhs[col];
43         }
44     }
45
46     // back substitution

```

```

47 Matrix x(n, 1, 0.0);
48 for (int i = n - 1; i >= 0; --i) {
49     double sum = rhs[i];
50     for (int j = i + 1; j < n; ++j)
51         sum -= a[i][j] * x(j, 0);
52     x(i, 0) = sum / a[i][i];
53 }
54 return x;
55 }

```

A.10 PortfolioOptimiser.h

```

1  #pragma once
2  #include "Matrix.h"
3  #include "LinearSolver.h"
4
5  // solves the Markowitz mean-variance optimisation problem
6  // minimise w'*Sigma*w subject to r_bar'*w = rP and e'*w = 1
7  //
8  // the KKT conditions give a (N+2) x (N+2) linear system which we solve
9  // using the Schur complement approach since the full KKT matrix is indefinite
10 class PortfolioOptimiser {
11 public:
12     PortfolioOptimiser(const Matrix& sigma,
13                       const Matrix& meanReturns,
14                       const LinearSolver& solver);
15
16     // returns the optimal weight vector for a given target return
17     Matrix optimise(double targetReturn) const;
18
19 private:
20     const Matrix& sigma_;
21     const Matrix& meanReturns_;
22     const LinearSolver& solver_;
23     int N_;
24 };

```

A.11 PortfolioOptimiser.cpp

```

1  #include "PortfolioOptimiser.h"
2  #include "ConjugateGradientSolver.h"
3  #include "LinearSystemSolver.h"
4  #include <stdexcept>
5
6  PortfolioOptimiser::PortfolioOptimiser(const Matrix& sigma,
7                                         const Matrix& meanReturns,
8                                         const LinearSolver& solver)
9      : sigma_(sigma), meanReturns_(meanReturns),
10        solver_(solver), N_(meanReturns.rows())
11 {}
12
13 Matrix PortfolioOptimiser::optimise(double targetReturn) const {
14     // the KKT system for Markowitz is:
15     // [ Sigma  -r_bar  -e ] [ w ] [ 0 ]
16     // [ -r_bar'  0      0 ] [ lam ] = [ -rP ]
17     // [ -e'      0      0 ] [ mu ] [ -1 ]
18     //

```

```

19 // Q is indefinite so we can't apply CG to the full system directly.
20 // instead use Schur complement: apply CG only to Sigma (positive definite)
21 // then solve a tiny 2x2 system for the Lagrange multipliers
22
23 Matrix e = onesVector(N_);
24
25 // solve Sigma*A = r_bar and Sigma*B = e using CG
26 Matrix A = solver_.solve(sigma_, meanReturns_);
27 Matrix B = solver_.solve(sigma_, e);
28
29 // build the 2x2 Schur system and solve for lambda and mu
30 Matrix S(2, 2);
31 S(0,0) = dot(meanReturns_, A);
32 S(0,1) = dot(meanReturns_, B);
33 S(1,0) = dot(e, A);
34 S(1,1) = dot(e, B);
35
36 Matrix rhs(2, 1);
37 rhs(0,0) = targetReturn;
38 rhs(1,0) = 1.0;
39
40 Matrix lagrange = LinearSystemSolver::solve(S, rhs);
41 double lambda = lagrange(0,0);
42 double mu      = lagrange(1,0);
43
44 // recover weights: w = lambda*A + mu*B
45 Matrix w(N_, 1, 0.0);
46 for (int i = 0; i < N_; ++i)
47     w(i,0) = lambda * A(i,0) + mu * B(i,0);
48
49 return w;
50 }

```

A.12 Backtester.h

```

1 #pragma once
2 #include "Matrix.h"
3 #include "LinearSolver.h"
4 #include <vector>
5 #include <string>
6
7 // holds the average OOS performance for one portfolio
8 struct BacktestResult {
9     double targetReturn; // target return used in optimisation
10    double oosReturn;     // average realised return out of sample
11    double oosVariance;   // average realised variance out of sample
12    double oosStdDev;     // square root of variance
13    double sharpeRatio;   // return / stddev (no risk free rate)
14    double maxDrawdown;   // worst peak to trough loss over all folds
15 };
16
17 // runs the rolling walk-forward backtest
18 // splits data into 100-day in-sample and 12-day out-of-sample windows
19 // slides forward by 12 days each time for 50 folds total
20 class Backtester {
21 public:
22     Backtester(const Matrix& allData,
23               const LinearSolver& solver,

```

```

24         int inSample = 100,
25         int oosDays  = 12);
26
27     // vanilla run using sample covariance
28     void run(const std::vector<double>& targetReturns);
29
30     // shrinkage run - uses  $\Sigma_{\text{shrink}} = (1-\delta)\Sigma + \delta F$ 
31     void runShrinkage(const std::vector<double>& targetReturns, double delta);
32
33     // returns averaged results across all folds
34     std::vector<BacktestResult> aggregateResults() const;
35
36     // equal weight 1/N benchmark for comparison
37     BacktestResult equalWeightBenchmark() const;
38
39     // saves per-fold results to a CSV file for plotting
40     void saveResultsToCSV(const std::string& filename) const;
41
42 private:
43     Matrix extractWindow(int startRow, int numRows) const;
44     void runImpl(const std::vector<double>& targetReturns, double shrinkageDelta);
45
46     const Matrix& allData_;
47     const LinearSolver& solver_;
48     int T_, N_;
49     int inSample_, oosDays_;
50     int numFolds_;
51
52     std::vector<double> targetReturns_;
53     std::vector<std::vector<double>> foldOosReturns_;
54     std::vector<std::vector<double>> foldOosVariances_;
55     std::vector<double> ewOosReturns_;
56     std::vector<double> ewOosVariances_;
57 };

```

A.13 Backtester.cpp

```

1  #include "Backtester.h"
2  #include "ParameterEstimator.h"
3  #include "PortfolioOptimiser.h"
4  #include <cmath>
5  // #include <fstream> -- replaced with FILE*
6  #include <iostream>
7  #include <stdexcept>
8
9  Backtester::Backtester(const Matrix& allData,
10                        const LinearSolver& solver,
11                        int inSample,
12                        int oosDays)
13      : allData_(allData),
14        solver_(solver),
15        T_(allData.rows()),
16        N_(allData.cols()),
17        inSample_(inSample),
18        oosDays_(oosDays),
19        numFolds_(0)
20  {
21      if (inSample_ <= 1)

```

```

22     throw std::invalid_argument("Backtester: in-sample size must be > 1");
23     if (oosDays_ <= 0)
24         throw std::invalid_argument("Backtester: out-of-sample size must be > 0");
25     if (T_ < inSample_ + oosDays_)
26         throw std::invalid_argument("Backtester: not enough data for one fold");
27 }
28
29 Matrix Backtester::extractWindow(int startRow, int numRows) const {
30     if (startRow < 0 || startRow + numRows > T_)
31         throw std::out_of_range("Backtester::extractWindow: index out of bounds");
32     Matrix w(numRows, N_);
33     for (int r = 0; r < numRows; r++)
34         for (int c = 0; c < N_; c++)
35             w(r, c) = allData_(startRow + r, c);
36     return w;
37 }
38
39 void Backtester::runImpl(const std::vector<double>& targetReturns,
40                         double shrinkageDelta) {
41     int numTargets = (int)targetReturns.size();
42
43     targetReturns_ = targetReturns;
44     foldOosReturns_.clear();
45     foldOosVariances_.clear();
46     ewOosReturns_.clear();
47     ewOosVariances_.clear();
48     numFolds_ = 0;
49
50     for (int start = 0;
51          start + inSample_ + oosDays_ <= T_;
52          start += oosDays_)
53     {
54         // In-sample: estimate parameters
55         Matrix isData = extractWindow(start, inSample_);
56         ParameterEstimator isEst(isData);
57         isEst.estimate();
58
59         const Matrix isMean = isEst.getMeanReturns();
60         const Matrix isCov = (shrinkageDelta > 0.0)
61                               ? isEst.getShrinkageCov(shrinkageDelta)
62                               : isEst.getCovMatrix();
63
64         // Out-of-sample: realised parameters
65         Matrix oosData = extractWindow(start + inSample_, oosDays_);
66         ParameterEstimator oosEst(oosData);
67         oosEst.estimate();
68         const Matrix oosMean = oosEst.getMeanReturns();
69         const Matrix oosCov = oosEst.getCovMatrix();
70
71         // Optimise using IN-SAMPLE parameters only
72         PortfolioOptimiser opt(isCov, isMean, solver_);
73
74         std::vector<double> foldRet(numTargets, 0.0);
75         std::vector<double> foldVar(numTargets, 0.0);
76
77         for (int t = 0; t < numTargets; ++t) {
78             Matrix w = opt.optimise(targetReturns[t]);
79
80             // OOS return:  $\bar{r}_{oos}^T * w$ 

```

```

81         double ret = 0.0;
82         for (int i = 0; i < N_; ++i)
83             ret += oosMean(i, 0) * w(i, 0);
84
85         // OOS variance:  $w^T * \text{Sigma}_{oos} * w$ 
86         double var = 0.0;
87         for (int i = 0; i < N_; ++i) {
88             double tmp = 0.0;
89             for (int j = 0; j < N_; ++j)
90                 tmp += oosCov(i, j) * w(j, 0);
91             var += w(i, 0) * tmp;
92         }
93
94         foldRet[t] = ret;
95         foldVar[t] = var;
96     }
97
98     foldOosReturns_.push_back(foldRet);
99     foldOosVariances_.push_back(foldVar);
100
101     // Equal-weight benchmark
102     const double ewW = 1.0 / N_;
103     double ewRet = 0.0, ewVar = 0.0;
104     for (int i = 0; i < N_; ++i)
105         ewRet += ewW * oosMean(i, 0);
106     for (int i = 0; i < N_; ++i)
107         for (int j = 0; j < N_; ++j)
108             ewVar += ewW * oosCov(i, j) * ewW;
109
110     ewOosReturns_.push_back(ewRet);
111     ewOosVariances_.push_back(ewVar);
112
113     ++numFolds_;
114 }
115
116 std::cout << "Backtest complete: " << numFolds_ << " folds.\n";
117 }
118
119 void Backtester::run(const std::vector<double>& targetReturns) {
120     runImpl(targetReturns, 0.0);
121 }
122
123 void Backtester::runShrinkage(const std::vector<double>& targetReturns,
124                             double delta) {
125     runImpl(targetReturns, delta);
126 }
127
128 std::vector<BacktestResult> Backtester::aggregateResults() const {
129     if (numFolds_ == 0)
130         throw std::runtime_error("Backtester: call run() before aggregateResults()");
131
132     int numTargets = (int)targetReturns_.size();
133     std::vector<BacktestResult> results(numTargets);
134
135     for (int t = 0; t < numTargets; ++t) {
136         double sumRet = 0.0, sumVar = 0.0;
137         for (int f = 0; f < numFolds_; ++f) {
138             sumRet += foldOosReturns_[f][t];

```

```

139         sumVar += foldOosVariances_[f][t];
140     }
141     const double avgRet = sumRet / numFolds_;
142     const double avgVar = sumVar / numFolds_;
143
144     // Maximum drawdown: peak cumulative return minus trough
145     double peak = 0.0, cumRet = 0.0, maxDD = 0.0;
146     for (int f = 0; f < numFolds_; ++f) {
147         cumRet += foldOosReturns_[f][t];
148         if (cumRet > peak) peak = cumRet;
149         double dd = peak - cumRet;
150         if (dd > maxDD) maxDD = dd;
151     }
152
153     BacktestResult& r = results[t];
154     r.targetReturn = targetReturns_[t];
155     r.oosReturn     = avgRet;
156     r.oosVariance   = avgVar;
157     r.oosStdDev     = std::sqrt(std::max(avgVar, 0.0));
158     r.sharpeRatio   = (r.oosStdDev > 1e-12) ? (avgRet / r.oosStdDev) : 0.0;
159     r.maxDrawdown   = -maxDD; // reported as negative (loss)
160 }
161 return results;
162 }
163
164 BacktestResult Backtester::equalWeightBenchmark() const {
165     if (numFolds_ == 0)
166         throw std::runtime_error("Backtester: call run() before
167             equalWeightBenchmark()");
168
169     double sumRet = 0.0, sumVar = 0.0;
170     double peak = 0.0, cumRet = 0.0, maxDD = 0.0;
171     for (int f = 0; f < numFolds_; ++f) {
172         sumRet += ewOosReturns_[f];
173         sumVar += ewOosVariances_[f];
174         cumRet += ewOosReturns_[f];
175         if (cumRet > peak) peak = cumRet;
176         double dd = peak - cumRet;
177         if (dd > maxDD) maxDD = dd;
178     }
179
180     BacktestResult r;
181     r.targetReturn = -1.0;
182     r.oosReturn     = sumRet / numFolds_;
183     r.oosVariance   = sumVar / numFolds_;
184     r.oosStdDev     = std::sqrt(std::max(r.oosVariance, 0.0));
185     r.sharpeRatio   = (r.oosStdDev > 1e-12) ? (r.oosReturn / r.oosStdDev) : 0.0;
186     r.maxDrawdown   = -maxDD;
187     return r;
188 }
189
190 void Backtester::saveResultsToCSV(const std::string& filename) const {
191     FILE* f = fopen(filename.c_str(), "w");
192     if (!f) {
193         fprintf(stderr, "Backtester: cannot open %s\n", filename.c_str());
194         return;
195     }
196     fprintf(f, "fold,target_return,oos_return,oos_variance\n");

```

```

197     int numTargets = (int)targetReturns_.size();
198     for (int fold = 0; fold < numFolds_; ++fold)
199         for (int t = 0; t < numTargets; ++t)
200             fprintf(f, "%d,%.8f,%.8f,%.8f\n",
201                     fold,
202                     targetReturns_[t],
203                     foldOosReturns_[fold][t],
204                     foldOosVariances_[fold][t]);
205
206     fflush(f);
207     fclose(f);
208     printf("Per-fold results saved to %s\n", filename.c_str());
209     fflush(stdout);
210 }

```

A.14 csv_reader.h

```

1  #pragma once
2  #include "Matrix.h"
3  #include <string>
4  #include <vector>
5
6  // reads asset returns from a CSV file
7  // returns a 2D vector of doubles (safe on Windows)
8  std::vector<std::vector<double>> loadCSVRaw(const std::string& filename);
9
10 // convenience wrapper that returns a Matrix directly
11 Matrix loadCSVMatrix(const std::string& filename);

```

A.15 csv_reader.cpp

```

1  #include "csv_reader.h"
2  #include <fstream>
3  #include <stdexcept>
4  #include <iostream>
5  #include <cstring>
6  #include <cstdlib>
7
8  std::vector<std::vector<double>> loadCSVRaw(const std::string& filename) {
9      FILE* f = fopen(filename.c_str(), "rb");
10     if (!f)
11         throw std::runtime_error("Cannot open: " + filename);
12
13     fseek(f, 0, SEEK_END);
14     long fsize = ftell(f);
15     fseek(f, 0, SEEK_SET);
16
17     char* buf = new char[fsize + 1];
18     if (fread(buf, 1, fsize, f) != static_cast<size_t>(fsize))
19         throw std::runtime_error("File read error: " + filename);
20     fclose(f);
21     buf[fsize] = '\0';
22
23     std::vector<std::vector<double>> rows;
24     rows.reserve(800);
25
26     char* p = buf;

```

```

27     char* end = buf + fsize;
28
29     while (p < end) {
30         char* lineEnd = p;
31         while (lineEnd < end && *lineEnd != '\n' && *lineEnd != '\r')
32             ++lineEnd;
33
34         std::vector<double> row;
35         row.reserve(90);
36
37         char* q = p;
38         while (q <= lineEnd) {
39             while (q < lineEnd && (*q == ' ' || *q == '\t')) ++q;
40
41             char* fieldEnd = q;
42             while (fieldEnd < lineEnd && *fieldEnd != ',') ++fieldEnd;
43
44             if (fieldEnd > q) {
45                 char tmp[64];
46                 int len = (int)(fieldEnd - q);
47                 if (len < 63) {
48                     memcpy(tmp, q, len);
49                     tmp[len] = '\0';
50                     char* endptr;
51                     double val = strtod(tmp, &endptr);
52                     if (endptr != tmp)
53                         row.push_back(val);
54                 }
55             }
56             q = fieldEnd + 1;
57         }
58
59         if (!row.empty())
60             rows.push_back(row);
61
62         while (lineEnd < end && (*lineEnd == '\n' || *lineEnd == '\r'))
63             ++lineEnd;
64         p = lineEnd;
65     }
66
67     delete[] buf;
68
69     if (rows.empty())
70         throw std::runtime_error("No data in: " + filename);
71
72     std::cout << "Loaded: " << rows.size() << " periods x "
73               << rows[0].size() << " assets\n";
74     std::cout.flush();
75     return rows;
76 }
77
78 // Fixed: construct Matrix directly on the stack -- no unnecessary heap allocation
79 Matrix loadCSVMatrix(const std::string& filename) {
80     auto rows = loadCSVRaw(filename);
81     int T = (int)rows.size();
82     int N = (int)rows[0].size();
83     Matrix data(T, N);
84     for (int t = 0; t < T; ++t)
85         for (int n = 0; n < N; ++n)

```

```

86         data(t, n) = rows[t][n];
87     return data;
88 }

```

A.16 main.cpp

```

1  // main.cpp
2  // Markowitz portfolio optimisation with rolling backtest
3  // Runs vanilla and shrinkage versions and prints results
4
5  #include <iostream>
6  #include <iomanip>
7  #include <vector>
8  #include <cmath>
9  #include <cstdio>
10 #include <cstdlib>
11 #include <stdexcept>
12
13 #include "csv_reader.h"
14 #include "Matrix.h"
15 #include "ConjugateGradientSolver.h"
16 #include "Backtester.h"
17
18 // creates 20 evenly spaced target returns from 0% to 10%
19 static std::vector<double> buildTargets(int n, double minT, double maxT) {
20     std::vector<double> t(n);
21     for (int i = 0; i < n; ++i)
22         t[i] = minT + i * (maxT - minT) / (n - 1);
23     return t;
24 }
25
26 // prints the side by side results table to the terminal
27 static void printTable(const std::vector<BacktestResult>& vanilla,
28                       const std::vector<BacktestResult>& shrink,
29                       const BacktestResult& ew) {
30     const int W = 13;
31     std::cout << "\n=== RESULTS (50 folds, per-period figures) ===\n";
32     std::cout << std::setw(10) << "Target%"
33               << std::setw(W) << "V.Ret%"
34               << std::setw(W) << "V.Std%"
35               << std::setw(W) << "V.Sharpe"
36               << std::setw(W) << "V.MaxDD%"
37               << std::setw(W) << "S.Ret%"
38               << std::setw(W) << "S.Std%"
39               << std::setw(W) << "S.Sharpe"
40               << std::setw(W) << "S.MaxDD%"
41               << "\n" << std::string(10 + W * 8, '-') << "\n";
42
43     std::cout << std::fixed << std::setprecision(4);
44     for (size_t t = 0; t < vanilla.size(); ++t) {
45         const auto& v = vanilla[t];
46         const auto& s = shrink[t];
47         std::cout << std::setw(9) << v.targetReturn * 100 << "%"
48                 << std::setw(W) << v.oosReturn * 100 << "%"
49                 << std::setw(W) << v.oosStdDev * 100 << "%"
50                 << std::setw(W) << v.sharpeRatio
51                 << std::setw(W) << v.maxDrawdown * 100 << "%"
52                 << std::setw(W) << s.oosReturn * 100 << "%"

```

```

53         << std::setw(W) << s.oosStdDev * 100 << "%"
54         << std::setw(W) << s.sharpeRatio
55         << std::setw(W) << s.maxDrawdown * 100 << "%"
56         << "\n";
57     }
58     std::cout << std::string(10 + W * 8, '-') << "\n";
59     std::cout << std::setw(9) << "EW BM"
60         << std::setw(W) << ew.oosReturn * 100 << "%"
61         << std::setw(W) << ew.oosStdDev * 100 << "%"
62         << std::setw(W) << ew.sharpeRatio
63         << std::setw(W) << ew.maxDrawdown * 100 << "%"
64         << "\n\n";
65     std::cout.flush();
66 }
67
68 // saves the aggregate results to a CSV file
69 // using FILE* instead of ofstream because ofstream crashes on Windows
70 static void writeAggregateCSV(const std::string& filename,
71                             const std::vector<BacktestResult>& vanilla,
72                             const std::vector<BacktestResult>& shrink,
73                             const BacktestResult& ew) {
74     FILE* f = fopen(filename.c_str(), "w");
75     if (!f) { std::cerr << "Cannot open " << filename << "\n"; return; }
76
77     fprintf(f, "method,target_return,oos_return,oos_variance,oos_stddev,
78             sharpe_ratio,max_drawdown\n");
79
80     for (const auto& r : vanilla)
81         fprintf(f, "vanilla,%.8f,%.8f,%.8f,%.8f,%.8f\n",
82             r.targetReturn, r.oosReturn, r.oosVariance,
83             r.oosStdDev, r.sharpeRatio, r.maxDrawdown);
84
85     for (const auto& r : shrink)
86         fprintf(f, "shrinkage,%.8f,%.8f,%.8f,%.8f,%.8f\n",
87             r.targetReturn, r.oosReturn, r.oosVariance,
88             r.oosStdDev, r.sharpeRatio, r.maxDrawdown);
89
90     fprintf(f, "benchmark,-1,%.8f,%.8f,%.8f,%.8f\n",
91         ew.oosReturn, ew.oosVariance, ew.oosStdDev,
92         ew.sharpeRatio, ew.maxDrawdown);
93
94     fflush(f);
95     fclose(f);
96     printf("Aggregate results saved to %s\n", filename.c_str());
97     fflush(stdout);
98 }
99
100 int main(int argc, char* argv[]) {
101     const std::string dataFile = (argc > 1) ? argv[1] : "asset_returns.csv";
102
103     // load the returns data
104     printf("Loading: %s\n", dataFile.c_str());
105     fflush(stdout);
106     Matrix* allData = new Matrix(loadCSVMatrix(dataFile));
107
108     // 20 target returns from 0% to 10%
109     const std::vector<double> targets = buildTargets(20, 0.0, 0.10);
110
111     // single CG solver shared by both backtests

```

```

111 ConjugateGradientSolver* cgSolver = new ConjugateGradientSolver(1e-6, 10000);
112
113 // run vanilla Markowitz backtest
114 printf("\nVanilla Markowitz...\n"); fflush(stdout);
115 Backtester* btVanilla = new Backtester(*allData, *cgSolver);
116 btVanilla->run(targets);
117 std::vector<BacktestResult> vanilla = btVanilla->aggregateResults();
118 BacktestResult ew = btVanilla->equalWeightBenchmark();
119 btVanilla->saveResultsToCSV("fold_results_vanilla.csv");
120 delete btVanilla;
121 printf("Vanilla done.\n"); fflush(stdout);
122
123 // run shrinkage Markowitz backtest (delta = 0.4 worked well in testing)
124 printf("\nShrinkage Markowitz (delta=0.4)...\n"); fflush(stdout);
125 Backtester* btShrink = new Backtester(*allData, *cgSolver);
126 btShrink->runShrinkage(targets, 0.4);
127 std::vector<BacktestResult> shrink = btShrink->aggregateResults();
128 btShrink->saveResultsToCSV("fold_results_shrinkage.csv");
129 delete btShrink;
130 printf("Shrinkage done.\n"); fflush(stdout);
131
132 // print and save results
133 printTable(vanilla, shrink, ew);
134 writeAggregateCSV("aggregate_results.csv", vanilla, shrink, ew);
135
136 printf("\nDone.\n"); fflush(stdout);
137
138 // using _Exit to avoid a crash on Windows during cleanup
139 _Exit(0);
140 }

```